DEEP LEARNING PROJECT 1

1. The Image as a Matrix

To you, a digit is a shape. To the computer, it is a grid of numbers.

- **The Input:** You have 28x28 pixels. That is 28 x 28 = 784 total pixels.
- **Pixel Intensity:** Each pixel is a number from **0 (Pitch Black)** to **255 (Pure White)**. Gray is somewhere in between.
- **The Logic:** We are feeding the computer a list of 784 numbers and asking it to find a mathematical correlation between those numbers and the label "7".

2. Train/Test Split

- **The Code:** `(x_train, y_train), (x_test, y_test) = mnist.load_data()`
- **The Logic:** You cannot test a student on the exact same questions used in class, or they will just memorize the answers.
    - **Training Set (60,000 images):** Used to adjust the weights of the model.
    - **Test Set (10,000 images):** Kept hidden in a "vault" until the very end to check if the model learned generic patterns or just *memorized* the training images.

3. Normalization

- **The Code:** `x_train / 255.0`
- **The Logic:** Neural networks struggle with large numbers (like 0 to 255). Large inputs require the computer to make massive mathematical jumps, which leads to instability.
- **The Fix:** By dividing by 255, we squash every number to be between **0.0 and 1.0**.
    - 255 becomes 1.0.
    - 0 becomes 0.0.
    - 127 becomes 0.5.
    - **Result:** The math flows much smoother (converges faster).

Phase 2: The Architecture (The Brain)

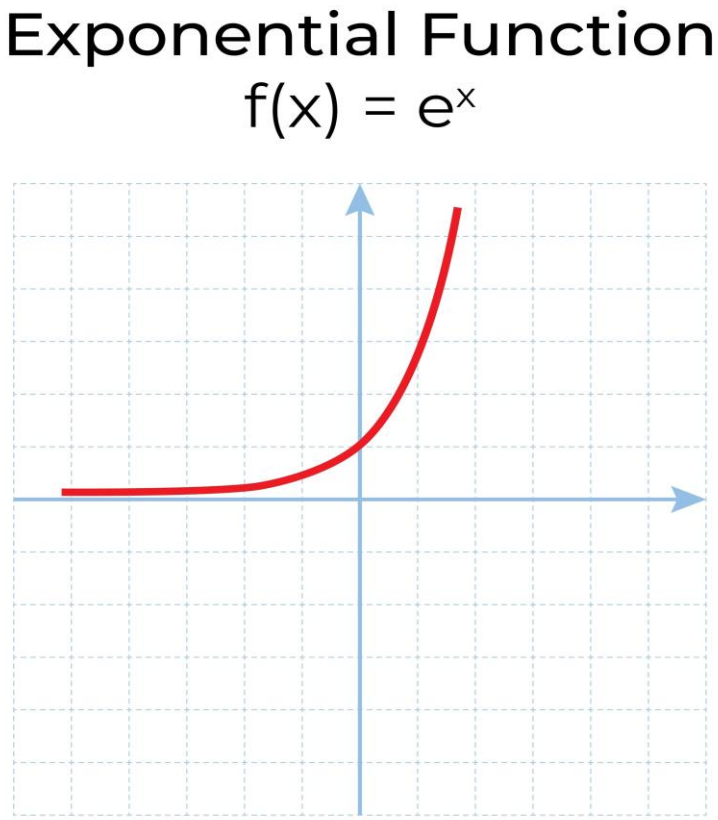
This is where we built the `Sequential` model. Think of this as an assembly line where data passes through stations.

Layer 1: Flatten

- **The Code:** `Flatten (input_shape= (28, 28))`
- **The Logic:** Our neural network is a standard "Dense" network. It cannot read 2D grids (squares). It only understands 1D lists (lines).
- **The Action:** It takes the 28 rows of pixels and stacks them end-to-end to create a single long line of **784 numbers**.
    - *Note:* This destroys the spatial information (it forgets that pixel [0,0] was above pixel [1,0]), which is why this model is good but not perfect.

Layer 2: Dense (The Hidden Layer)

- **The Code:** `Dense (128, activation='relu')`
- **The Logic:** This is where the thinking happens. You created **128 Artificial Neurons**.
- **What is a Neuron?** A neuron is simply a mathematical function. Each neuron connects to all 784 input pixels. It has its own set of **Weights (w)** and a **Bias (b)**.
    - **The Equation: z = (x_1.w_1) + (x_2 .w_2) + ... + b**
    - If a pixel has a high value and the weight is high, the neuron gets "excited."
- **The Activation Function: ReLU (Rectified Linear Unit)**



Shutterstock

Explore

Why do we need it? Without an activation function, the neural network is just linear algebra (a straight line). It could never learn curves (like the loop in a '9' or a '6').

The Logic: ReLU is a simple gatekeeper.

    If the calculated value is Negative, it outputs 0 (Neuron stays silent).

    If the calculated value is Positive, it passes it through unchanged.

Result: This introduces "non-linearity," allowing the model to learn complex shapes.

Layer 3: Dropout

- **The Code:** `Dropout (0.2)`
- **The Logic:** This is a technique to prevent **Overfitting**.
- **The Metaphor:** Imagine a classroom where one student is a genius and everyone else just copies them. The class average is high, but if the genius is sick, everyone fails.

- **The Action:** In every round of training, we randomly "knock out" 20% of the neurons (set them to zero). This forces the *other* neurons to work harder and learn the patterns independently. It builds redundancy.

Layer 4: Output Layer

- **The Code:** `Dense (10, activation='softmax')`
- **The Logic:**
    - **Why 10?** Because we have 10 possible categories (digits 0-9).
    - **Why Softmax?** The previous layers output raw numbers (e.g., -5.4, 12.1, 3.2). These are hard to interpret. **Softmax** forces these numbers to become **probabilities** that sum up to 100%.
    - *Example Output:* `[0.01, 0.90, 0.09 ...]` -> "I am 90% sure this is a '1'."

Phase 3: Compilation (The Learning Strategy)

You built the car; now you must tell it how to drive.

1. Loss Function: Sparse Categorical Crossentropy

- **The Logic:** This measures **"How wrong is the model?"**
- If the image is a "5" and the model predicts "5" with 90% confidence, the Loss is **Low**.
- If the image is a "5" and the model predicts "3" with 80% confidence, the Loss is **High**.
- The goal of training is to make this number as close to 0 as possible.

2. Optimizer: Adam

- **The Logic:** This is the mechanic that fixes the engine based on the Loss score.
- **Backpropagation:** When the model makes a mistake, the Optimizer looks at the math and calculates which specific weights (w) contributed to the error.
- **Gradient Descent:** It tweaks those weights slightly in the opposite direction to reduce the error next time.
- **Why Adam?** It stands for "Adaptive Moment Estimation." It is a smart algorithm that adjusts the learning speed automatically. It learns fast at the beginning and slows down for precision at the end.

Phase 4: Training (The Loop)

- **The Code:** `model.fit(..., epochs=5)`
- **The Logic:**
    - **Forward Pass:** The model looks at a batch of images and makes guesses.
    - **Loss Calculation:** It compares guesses to the real answers (`y_train`) and calculates the error score.
    - **Backward Pass (Backprop):** The Optimizer updates the weights of the 128 neurons to fix the error.
    - **Repeat:** It does this for all 60,000 images. That is **Epoch 1**.
    - It repeats the whole process 5 times.

Summary of the Flow

1. **Input:** 784 numbers (Pixels).
2. **Dense Layer:** 128 Neurons multiply inputs by weights (w) and add bias (b).
3. **ReLU:** Turns negative values to 0 (adds complexity).
4. **Dropout:** Randomly silences neurons (forces robustness).
5. **Output Layer:** 10 Neurons produce raw scores.
6. **Softmax:** Converts scores to probabilities (e.g., "97% chance it's a 7").
7. **Optimizer:** Updates the w and b based on how wrong the guess was.